Neominal Akademi

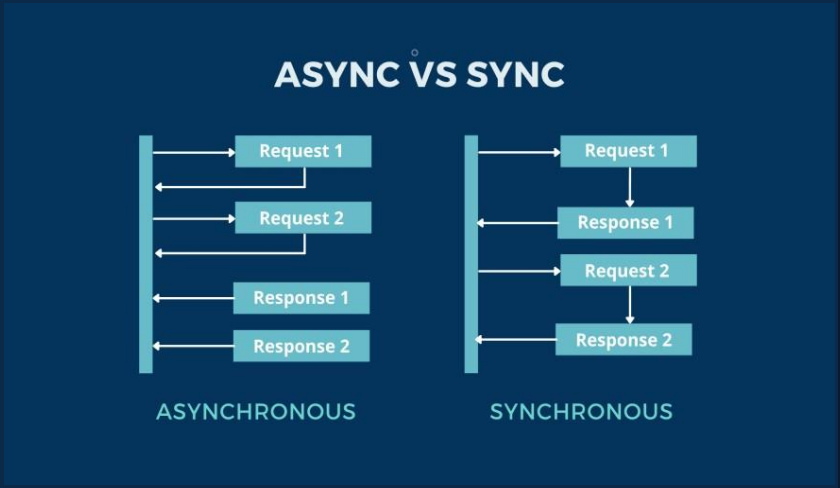
Asenkron ve Paralel Programlama

Sürüm / Tarih

v1.0 • Mart 2026

Asenkron ve Paralel Programlama

Canlı oturumlarda eğitmenin yararlanacağı kaynak.



Hedef Kitle

- Asenkron ve Multi-Thread Programlama
- Thread Pool Mekanizması
- Thread ve Task Arasındaki Farklar

Bu sunum, canlı oturumlarda eğitime destek sağlamak amacıyla Neominal Akademi tarafından hazırlanmıştır. Tüm hakları Neominal tarafından saklıdır. Yazılı izin olmaksızın dağıtılması ve çoğaltılması yasaktır.

Eğitim İçeriği

Asenkron & Paralel Programlama (.NET)

Ölçeklenebilir ve güvenli uygulamalar için **eşzamanlılık** temellerini öğrenin: asenkron akış, multi-thread yaklaşımı, Thread Pool ve Task modeli.

Öğrenme hedefleri (teorik)

- Asenkron programlama: I/O beklerken thread'i bloklamadan akışı yönetme (ölçeklenebilirlik).
- Multi-thread: aynı anda çalışan thread'lerle paralel iş yürütme (eşzamanlılık / paralellik ayrımı).
- Thread Pool: kısa ömürlü işler için paylaşımlı worker havuzu; yeniden kullanım, daha düşük maliyet.
- Task modeli: Task/Task<T> ile iptal, hata yönetimi ve kompozisyon (WhenAll/WhenAny) yaklaşımı.

Temel ayrım (özet)

Asenkron ≠ Multi-thread

Asenkron: "bekleme"yi verimli yönetir (thread serbest kalır).
Multi-thread: "aynı anda çalışma"yı artırır (çok çekirdek kullanımı).

Thread vs Task

Thread: düşük seviye, yönetimi zor, maliyeti yüksek.
Task: modern soyutlama; Thread Pool üstünde planlanır, kompozisyon ve iptal desteği güçlü.

Odak: kavramları doğru konumlandırma • Sonraki slaytlar: Asenkron vs Multi-thread, Thread Pool, Task modeli, Paralel programlama, Senkronizasyon

Temelden ileri seviyeye akış: Asenkron → Thread Pool → Task → Parallel → Senkronizasyon

Amaç: “Eşzamanlılık” (concurrency) ve “paralellik” (parallelism) ayrımını kavrayıp doğru aracı doğru problem için seçmek. Bu yol, kavramları birbirinin üzerine inşa eder.

1

Asenkron
Concurrency

Bloksuz bekleme ile aynı thread üzerinde ilerleme. Özellikle I/O-bound işlerde ölçeklenebilirlik sağlar.

Ana fikir: “Beklerken thread boşa çıkmalı”

2

Thread Pool
Paylaşımlı kaynak

Runtime’ın yönettiği hazır thread’ler havuzu. Yeni thread yaratma maliyetini azaltır ve işleri sıraya alır.

Amaç: verimli planlama + daha az overhead

3

Task Model
async/await

“İş”i temsil eden soyutlama: iptal, hata ve sonuç yönetimi standarttır. Genellikle Thread Pool üzerinde çalışır.

Mesaj: Task ≠ Thread (çoğu zaman “planlanan iş”)

4

Parallel
Multi-core

CPU-bound iş yükünü birden çok çekirdeğe dağıtma. Hedef: daha kısa toplam çalışma süresi.

Not: I/O için değil, işlemci için uygundur

5

Senkron
Güvenlik

Paylaşılan veriyi koruma: yarış durumlarını önleme, doğru kilit seçimi, görünürlük ve atomiklik.

Son hedef: doğruluk + stabilite

Kavram kontrol listesi (bu slayttan sonra)

Asenkronluk “çok thread” demek değildir • Thread Pool “sınırsız thread” değildir • Paralellik performans garantisi vermez • Senkronizasyon performans/doğruluk dengesi gerektirir

Tercih rehberi (hızlı)

I/O → Asenkron • CPU → Parallel • Paylaşılan state → Senkronizasyon

Asenkron Programlama vs Multi-Thread Programlama

Biri beklerken işi bırakıp devam etmeyi (I/O), diğeri aynı anda iş yürütmeyi (CPU) hedefler — çoğu sistemde birlikte kullanılır.

Asenkron Programlama

`async/await • I/O bekleme yönetimi`

Özet

Temel fark: "Beklerken bloklama" yerine devam etme (non-blocking).

Çalışma prensibi: İş, I/O tamamlanınca devam (continuation) ile sürer; thread boşa çıkar.

Kullanım alanı: HTTP, veritabanı, dosya, kuyruk gibi I/O-bound işler.

Avantaj: Daha az thread ile daha çok isteğe yanıt → ölçeklenebilirlik.

Dezavantaj: Akış takibi, context yakalama, hatalı bloklama (Wait/Result) ile sorun çıkabilir.

Ne zaman: "Bekleme" baskınsa ve thread'i meşgul etmek istemiyorsan.

Multi-Thread Programlama

`Thread/Task • eşzamanlı yürütme`

Özet

Temel fark: Birden fazla işin aynı anda yürütülmesi (concurrency / parallelism).

Çalışma prensibi: İşler farklı thread'lerde koşar; CPU çekirdekleri arasında paylaştırılır.

Kullanım alanı: Sıkı hesaplama, görüntü işleme, veri dönüştürme gibi CPU-bound işler.

Avantaj: Çok çekirdeği kullanarak throughput ve toplam süreyi iyileştirme.

Dezavantaj: Race condition, deadlock, kilit maliyeti, context switch ve debug zorluğu.

Ne zaman: CPU süresi baskınsa ve iş parçalanabiliyorsa (paralelleştirilebilir).

Karıştırma noktası (önemli)

- Asenkronluk "paralel çalışmak" demek değildir; tek thread ile de asenkron akış mümkündür.
- Multi-threading, hızlanma sağlayabilir; fakat paylaşılan state varsa senkronizasyon zorunludur.

Pratik kural

I/O bekleme → asenkron akış. CPU hesap → paralelleştirme (gerekirse multi-thread).

Thread Pool Mekanizması (Teorik)

Thread oluşturma maliyetini azaltmak için .NET, işleri paylaşımlı işçi thread havuzunda çalıştırır. `Task` ve `Parallel` çoğunlukla bu havuzu kullanır.

Thread Pool nedir?

OS thread'lerini yeniden kullanarak hızlı ve ölçeklenebilir yürütme

Çalışma prensibi (özet)

- Havuzda hazır bekleyen **worker thread**'ler vardır
- İş bir `work queue`'ya eklenir; uygun thread işi alır
- Thread'ler genellikle **background** çalışır; süreç kapanınca beklenmez
- Runtime, yük arttıkça thread sayısını **kademeli** büyütür (aşırı artışı sınırlamaya çalışır)
- Amaç: **düşük başlatma maliyeti + daha iyi throughput**

Zihinsel model: "Kuyruk + işçi thread'ler"

İş (work item) → Work Queue → Worker Thread → Çalıştır

Not: Uzun süre bloklayan işler (özellikle I/O'yu senkron beklemek) havuz thread'lerini meşgul ederek **thread starvation** riskini artırabilir.

Avantajlar, kullanım ve pratik ipuçları

"Havuz dostu" kod: kısa, non-blocking, iptal edilebilir

Best practices

- **I/O beklemelerini** senkron bloklamak yerine `async/await` kullanın
- Uzun CPU işleri için paralelliği **sınırlandırın** (aşırı paralellik thrash yaratır)
- **Cancellation** tasarlayın (token ile iptal edilebilir iş akışları)
- Kritik kaynakları havuz thread'lerinde **uzun süre** kilit altında tutmayın
- "Fire-and-forget" işlerde hata kaybolmasın: **log/izleme** ve yaşam döngüsü tasarlayın
- Havuz thread'lerine **thread-affine** (UI thread gibi) beklentiler yüklemeyin

Ne zaman "dedicated thread"? Sürekli çalışan, özel öncelik/affinity isteyen veya izolasyon gerektiren işler.

Kısa özet: **Thread Pool** verim için "yeniden kullanım" sağlar; performans sorunlarının çoğu **bloklama** ve **kontrolsüz paralellikten** doğar.

Thread vs Task: Temel Farklar (Teorik)

Thread OS iş parçacığıdır; Task ise işin tamamlanmasını temsil eden, genelde Thread Pool üzerinde çalışan daha üst seviye bir soyutlamadır.

Thread

`OS thread • daha düşük seviye kontrol`

Öne çıkanlar

Oluşturma maliyeti: Daha yüksek; her thread ayrı stack/OS kaynağı tüketir.

Zamanlama: OS scheduler tarafından yürütülür; kontrol daha "ham" ve ayrıntılıdır.

Yaşam döngüsü: Start/Join/IsBackground gibi süreçleri doğrudan yönetirsiniz.

Değer döndürme: Doğrudan sonuç modeli yoktur; tasarım ekstra mekanizma ister.

İptal & hata: İptal ve exception yönetimi genellikle manuel kurgulanır.

Ne zaman: Uzun ömürlü/dedike işler, özel öncelik/arka plan davranışı gibi senaryolar.

Task

`iş (work) soyutlaması • async/await ile uyumlu`

Öne çıkanlar

Oluşturma maliyeti: Daha düşük; çoğunlukla Thread Pool ile yeniden kullanım.

Zamanlama: Runtime planlar; work-stealing vb. stratejilerle verimli dağıtım hedeflenir.

Yaşam döngüsü: "Başladı/bitti/iptal/hata" durumları standarttır; composition kolaydır.

Değer döndürme: Task<T> ile doğal sonuç modeli.

İptal & hata: CancellationToken ve await + try/catch ile tutarlı yönetim.

Ne zaman: Varsayılan tercih. I/O beklemeli akışlar ve bağımsız işleri compose etmek için.

Seçim rehberi (kısa)

- **Uygulama kodunun %80'i:** Task + async/await ile daha güvenli ve okunaklı.
- **Thread:** özel yaşam döngüsü/arka plan davranışı veya izole çalışma gerektiğinde.

Kritik not

Task "yeni thread" demek değildir. Çoğu zaman amaç: thread'i bloklamadan işi yönetmek ve ölçeklenmek.

Senkronizasyon Mekanizmaları & Best Practices

Amaç: **doğruluk** (race condition yok), **performans** (gereksiz kilit yok) ve **bakım kolaylığı** (doğru araç, doğru yerde).

Temel kavramlar ve araçlar

Paylaşımlı state'i koru: atomiklik • görünürlük • kritik bölge

Race condition (neden olur?)

Paylaşımlı veri birden çok thread tarafından aynı anda güncellenirse "kaybolan güncelleme" ve tutarsız sonuçlar oluşur. Özellikle **okuma → değiştirme → yazma** adımlı işlemler korunmalıdır.

Araçlar (ne çözer?)

- `lock` / `Monitor`: aynı process içinde kritik bölge
- `Mutex`: process'ler arası kilitleme (named)
- `SemaphoreSlim`: eşzamanlı N erişim / async uyumlu seri erişim
- `Interlocked`: sayısal/tek adımlı atomik işlemler
- `volatile`: görünürlük sağlar, **atomiklik sağlamaz**

Hızlı seçim ipuçları (kısa rehber)

- Async akışta kritik bölge: `SemaphoreSlim(1,1)`
- Cross-process koruma: `Mutex`
- Basit sayaç/flag: `Interlocked` (en hızlı)
- Timeout / sinyalleşme ihtiyacı: `Monitor` (TryEnter / Wait-Pulse)

Best practices

Stabilite için: bloklama azalt • paralelliği yönet • kaynakları garantiyle bırak

Pratik kurallar

- Senkron bekleme yerine **asenkron I/O** tercih et (thread'leri meşgul etme)
- Kilit kapsamını **küçük** tut; uzun iş yapma, dış çağrı yapma
- **İptal** ve timeout stratejisi tasarla (özellikle uzun işlerde)
- Paylaşımlı state'i azalt: immutability, mesajlaşma, izolasyon
- Paralellikte "daha çok" her zaman "daha hızlı" değildir: **dereceyi sınırla**
- Kaynak bırakmayı garanti et: her zaman `finally`

Yaygın hatalar

- "Sadece `volatile` yeter" yanılgısı (atomiklik değil)
- Kilit altında bekleme / I/O / uzun CPU işi → tıkanma
- Kilidi bırakmayı unutma (release/exit) veya exception'ı yutma
- Fire-and-forget" işlerde gözlenmeyen hata ve yaşam döngüsü problemi

Kısa özet: Eşzamanlılıkta risk genellikle **paylaşımlı state** ve **bloklamadan** gelir; doğru kilit seçimi + kısa kritik bölge çoğu problemi çözer.